

DSA Arrays Day1 - Basics

ITSRUNTYM

- **1. Introduction to DSA**

What is DSA?

DSA stands for **Data Structures and Algorithms**:

1. **Data Structures (DS)**: A way to organize, store, and manage data efficiently (e.g., Arrays, LinkedLists, Stacks).
 2. **Algorithms (A)**: A finite set of instructions or logic used to perform a specific task (e.g., Searching, Sorting).
- **DS** is about *how* data is stored,
A is about *how* we manipulate or process that data.

Why Learn DSA?

• Benefit	• Explanation
• Problem Solving	• Improves logical thinking and ability to solve real-life problems.
• Coding Interviews	• Essential for cracking coding rounds in top tech companies.
• Optimized Code	• Helps write code that is both memory and time efficient.
• Real-world Applications	• Used in databases, operating systems, web servers, search engines, etc.
•	
Real-Life Analogies	
• Concept	• Analogy
• Array	• Bookshelf with slots (each slot = index)
• Stack	• Plates in cafeteria (Last-In-First-Out)

• Queue	• People standing in line (First-In-First-Out)
• HashMap	• Dictionary with word-definition pairs
• Tree	• Company hierarchy or file system
• Graph	• Google Maps, Facebook connections

2. Types of Data Structures

Linear Data Structures

Data elements are arranged in a sequential manner.

- **Array** – Fixed-size, contiguous memory blocks.
- 1. **Linked List** – Dynamic memory, each element points to the next.
- 1. **Stack** – LIFO structure (e.g., undo operation).
- 1. **Queue** – FIFO structure (e.g., task scheduling).
-

Non-Linear Data Structures

Data elements are not arranged linearly.

- 1. **Tree** – Hierarchical (e.g., XML/HTML parsing, binary search trees).
- 1. **Graph** – Set of connected nodes (e.g., social networks, maps).
-

Hashing

- 1. **HashMap / HashSet** – Store key-value pairs, provide average **O(1)** operations using **hash functions**.

Other Advanced Structures

- 1. **Heap** – Special tree-based structure used in priority queues.
- 1. **Trie** – Prefix tree, used for autocomplete and dictionary word matching.

-

3. Algorithm Basics

What is an Algorithm?

An **algorithm** is a step-by-step procedure or formula to solve a problem.

Example 1: Add two numbers

Step 1: Input two numbers

Step 2: Add them

Step 3: Return the result

Example 2: Binary Search

1. Search in a sorted array by repeatedly dividing the array in half.

-

How to Analyze an Algorithm?

We analyze algorithms based on:

1. **Time Complexity** – How much time it takes to run.
2. **Space Complexity** – How much memory it consumes.

-

4. Time and Space Complexity

What is Time Complexity?

1. Measures **how the execution time increases** with input size n .

• Code	• Time Complexity
• for ($i = 0; i < n; i++$)	• $O(n)$
• for ($i = 0; i < n; i++$) for ($j = 0; j < n; j++$)	• $O(n^2)$

What is Space Complexity?

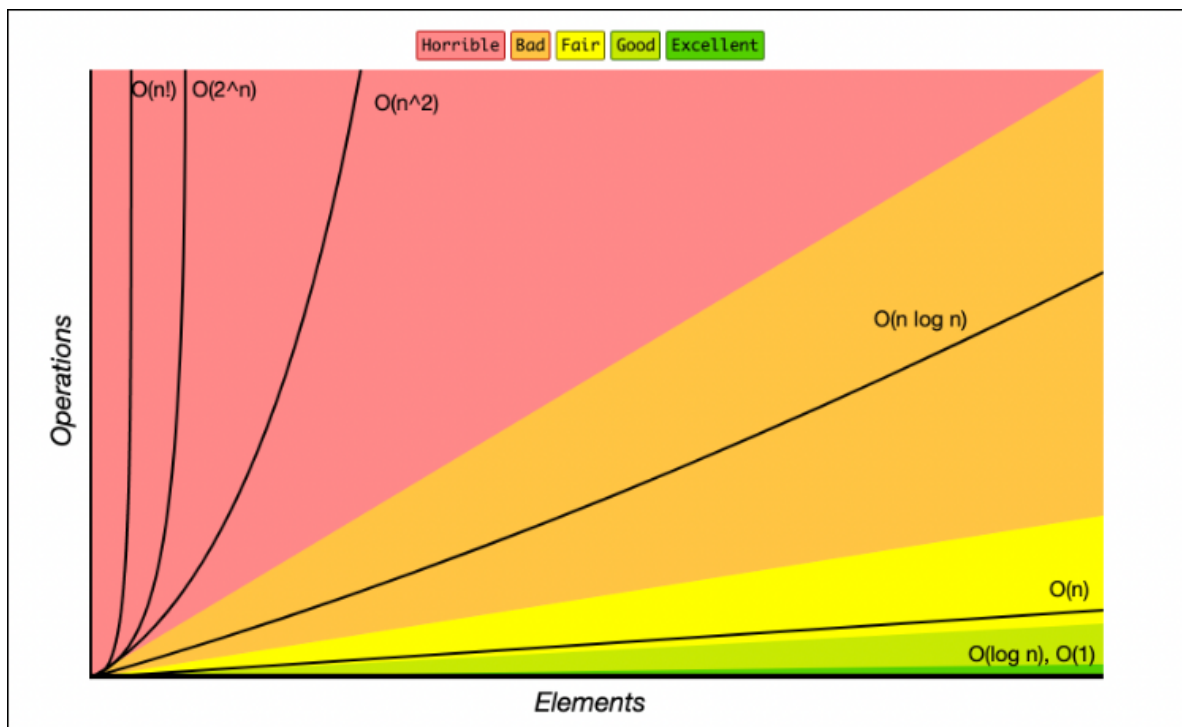
- Measures **how memory usage grows** with input size n .
-

Big O Notation

Big O notation is a mathematical notation that describes the limiting behavior of a function when the argument tends towards a particular value or infinity, often used to analyze the efficiency of algorithms. It focuses on how the runtime or space usage of an algorithm grows as the input size increases

Big-O tells the **upper bound** of time/space required.

• Notation	• Name	• Example
• $O(1)$	• Constant	• Accessing array element
• $O(\log n)$	• Logarithmic	• Binary search
• $O(n)$	• Linear	• Traversing an array
• $O(n \log n)$	• Linearithmic	• Merge sort, Quick sort
• $O(n^2)$	• Quadratic	• Bubble sort, nested loops
• $O(2^n)$	• Exponential	• Recursion in Fibonacci
• $O(n!)$	• Factorial	• Solving permutations



Best, Worst, and Average Case

• Case	• Description
• Best	• Minimum time (ideal input)
• Worst	• Maximum time (most complex input)
• Average	• Expected time for random input

Why Ignore Constants in Big O?

We focus on **scalability**, not machine-specific speed.

$O(2n)$ is still **$O(n)$** because constants don't affect growth rate.

Big-O Notation Examples

$O(1)$ – Constant Time

Accessing array element or basic arithmetic operation

```
int[] arr = {1, 2, 3, 4, 5};  
System.out.println(arr[2]); // Always takes same time
```

Explanation: No matter the size of the array, accessing an index takes constant time.

$O(\log n)$ – Logarithmic Time

Binary Search

```
int binarySearch(int[] arr, int key) {  
    int low = 0, high = arr.length - 1;  
    while (low <= high) {  
        int mid = (low + high) / 2;  
        if (arr[mid] == key) return mid;  
        else if (arr[mid] < key) low = mid + 1;  
        else high = mid - 1;  
    }  
    return -1;  
}
```

Explanation: The array is divided by 2 in each step $\Rightarrow \log_2(n)$ steps.

- The sequence can be represented as
- $n, n/2, n/4, \dots, n/2^{k-1}$

$$n/2^{k-1} = 1, \quad n = 2^{k-1}$$

$O(n)$ – Linear Time

Traversing an array

```
int sum = 0;  
for (int i = 0; i < arr.length; i++) {  
    sum += arr[i];  
}
```

Explanation: Every element is visited once. Time grows linearly with input size.

$O(n \log n)$ – Linearithmic Time

Merge Sort or Quick Sort

```
void mergeSort(int[] arr, int l, int r) {  
    if (l < r) {  
        int mid = l + (r - l) / 2;  
        mergeSort(arr, l, mid);  
        mergeSort(arr, mid + 1, r);  
        merge(arr, l, mid, r);  
    }  
}
```

Explanation: Divides array in $\log(n)$ levels, each level does $O(n)$ work
→ $n * \log n$.

$O(n^2)$ – Quadratic Time

Bubble Sort or nested loops

```
for (int i = 0; i < n; i++) {  
    for (int j = 0; j < n; j++) {  
        System.out.print("*");  
    }  
}
```

Explanation: Two nested loops. If $n = 5$, it runs 25 times (5×5) ⇒ $O(n^2)$.

$O(2^n)$ – Exponential Time

Recursive Fibonacci

```
int fib(int n) {  
    if (n <= 1) return n;  
    return fib(n - 1) + fib(n - 2);  
}
```

Explanation: Each call makes 2 more calls → grows like a binary tree

of height n .

At each level of the recursion tree:

1. Level 0: 1 call ($\text{fib}(n)$)
2. Level 1: 2 calls ($\text{fib}(n-1)$, $\text{fib}(n-2)$)
3. Level 2: 4 calls
4. Level 3: 8 calls
5. ...
6. Level k : 2^k calls

•

$O(n!)$ – Factorial Time **Generating all permutations**

```
void permute(String str, String res) {  
    if (str.length() == 0) {  
        System.out.println(res);  
        return;  
    }  
    for (int i = 0; i < str.length(); i++) {  
        char ch = str.charAt(i);  
        String rem = str.substring(0, i) + str.substring(i + 1);  
        permute(rem, res + ch);  
    }  
}
```

Explanation: For string of length n , it generates $n!$ permutations.

Simple Example: "ABC"

Step 1: permute("ABC", "")

1. Picks 'A', calls: $\text{permute}(\text{"BC"}, \text{"A"})$
2. Picks 'B', calls: $\text{permute}(\text{"AC"}, \text{"B"})$
3. Picks 'C', calls: $\text{permute}(\text{"AB"}, \text{"C"})$

- Let's expand the first one:

Step 2: permute("BC", "A")

1. Picks 'B', calls: $\text{permute}(\text{"C"}, \text{"AB"})$
2. Picks 'C', calls: $\text{permute}(\text{"B"}, \text{"AC"})$

- **Step 3: permute("C", "AB")**
1. Only 'C' left → calls permute("", "ABC") → **prints: "ABC"**
- **Step 3: permute("B", "AC")**
1. Only 'B' left → calls permute("", "ACB") → **prints: "ACB"**
-

Same steps happen for "B" and "C" chosen in Step 1:

You'll get:

1. "ABC"
 2. "ACB"
 3. "BAC"
 4. "BCA"
 5. "CAB"
 6. "CBA"
- Total: **6 permutations = 3! = 6**

Time Complexity Analysis

Let's analyze it **mathematically**:

For a string of length n , say permute(n):

1. At first level: n recursive calls (each character picked once)
 2. At second level: $n-1$ calls for each → total $n \times (n-1)$
 3. At third level: $n \times (n-1) \times (n-2)$
 4. ...
 5. Until 1
- $T(n) = n \times (n-1) \times (n-2) \times \dots \times 1 = n!$

Hence, the **time complexity is $O(n!)$** .

Also, each of these calls takes **$O(n)$** time to build new strings using substring() and res + ch, so:

Summary Table with Code Reference

• Notation	• Real-life Code Task	• Time Grows As
• $O(1)$	• Accessing arr[i]	• Constant
• $O(\log n)$	• Binary Search	• Halving each time

• $O(n)$	• Sum of array	• Single loop
• $O(n \log n)$	• Merge Sort	• Divide and conquer
• $O(n^2)$	• Nested loop pattern	• Grid-like operations
• $O(2^n)$	• Recursive Fibonacci	• Binary tree growth
• $O(n!)$	• All permutations	• Brute-force with all orderings

Activities

Sum of Array Elements

```
for (int i = 0; i < n; i++) {
    sum += arr[i];
}
// O(n)
```

Nested Loops Example

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        // some operation
    }
}
// O(n^2)
```

Binary Search

```
int binarySearch(int[] arr, int key) {
    int low = 0, high = arr.length - 1;
    while (low <= high) {
        int mid = (low + high) / 2;
        if (arr[mid] == key) return mid;
        else if (arr[mid] < key) low = mid + 1;
    }
}
```

```

        else high = mid - 1;
    }
    return -1;
}
// O(log n)

```

5. Introduction to Arrays (in-depth)

What is an Array?

An **array** is a **collection of elements** (values) of the **same data type**, stored in **contiguous memory locations**.

- Each element in an array is identified by an **index**.
1. Indexing starts from 0 → The first element is at arr[0].

•

Analogy:

Think of an array like a **row of lockers**, each locker has:

1. A **unique number (index)** on the front
2. A **value (element)** stored inside

•

Memory Layout of Array

```
arr = [10, 20, 30, 40, 50]
```

```
// Memory-wise:
```

```
Index:  0  1  2  3  4
```

```
Value:  10 20 30 40 50
```

```
Address: 100 104 108 112 116 (if int = 4 bytes)
```

1. Every element is placed at a **fixed distance** from the previous one.

2. This enables **O(1) random access** using

-

Characteristics of Arrays

• Property	• Explanation
• Fixed Size	• Once created, size cannot change.
• Contiguous Memory	• Elements are stored next to each other in memory.
• Zero-based Indexing	• First element is at index 0.
• Same Type Elements	• All elements must be of the same data type.
• Fast Access (O(1))	• Direct access via index.
• Slow Insertion/Deletion	• Requires shifting elements (O(n)).
•	

Strengths of Arrays

• Strength	• Why it matters
• Simple	• Easy to declare, initialize, and use.
• Efficient Access	• Access any element in constant time.
• Low Overhead	• Memory overhead is low compared to dynamic structures.
•	

Limitations of Arrays

• Limitation	• Impact
• Fixed Size	• Cannot grow/shrink after creation.
• Costly Insert/Delete	• All elements must shift if not at end.
• No built-in methods	• Manual implementation of common operations.
• Wastage	• If over-allocated for safety, unused memory is wasted.
•	

Real-Life Uses of Arrays

• Scenario	• Description
• Leaderboard	• Store top player scores in order.
• Image Data	• 2D array for pixel matrix (RGB values).
• Music Playlist	• Storing songs to be played in order.
• Sensor Logs	• Recording a sequence of readings.
• Matrix Calculations	• Used in scientific computation.

1. Declaration of Arrays

```
int[] arr1;    // Recommended
int arr2[];    // Also valid
```

2. Initialization with Size

```
int[] arr = new int[5]; // array of size 5, default values are 0
```

3. Initialization with Values (Inline)

```
int[] arr = {10, 20, 30, 40, 50};
```

4. Initialization using new with Values

```
int[] arr = new int[] {1, 2, 3, 4};
```

Note: Cannot do `int[] arr; arr = {1, 2, 3};` – that's invalid.

5. Access and Modify Elements

```
arr[0] = 100;           // set value
System.out.println(arr[0]); // get value
```

6. Array Length

```
System.out.println(arr.length); // Not a method – it's a property
```

7. Looping Over Arrays

a. For Loop

```
for (int i = 0; i < arr.length; i++) {
    System.out.println(arr[i]);
}
```

b. Enhanced For Loop (for-each)

```
for (int num : arr) {
    System.out.println(num);
}
```

8. Sorting Arrays

```
import java.util.Arrays;

Arrays.sort(arr); // Sorts in ascending order
```

9. Copying Arrays

```
int[] copy = Arrays.copyOf(arr, arr.length);
```

10. Multidimensional Arrays (2D Arrays)

```
int[][] matrix = {
    {1, 2},
    {3, 4}
};
```

```
System.out.println(matrix[1][0]); // Output: 3
```

11. Printing Arrays

```
System.out.println(Arrays.toString(arr));    // 1D
System.out.println(Arrays.deepToString(matrix)); // 2D
```

12. Returning Arrays from Methods

```
public static int[] getArray() {
    return new int[] {1, 2, 3};
}
```

13. Array of Objects

```
String[] names = new String[] {"Alice", "Bob", "Charlie"};
```

Common Array Operations

• Operation	• Time	• Notes
• Access arr[i]	• O(1)	• Direct index access
• Traverse	• O(n)	• Loop over elements
• Insert at End	• O(1)	• Only if space available
• Insert at Index	• O(n)	• Need to shift elements
• Delete	• O(n)	• Shift elements left
• Search (Linear)	• O(n)	• Compare each
• Search (Binary)	• O(log n)	• Array must be sorted

Array vs ArrayList

What is an ArrayList?

- ArrayList is a **resizable array** from **Java's Collections Framework**.
 1. Internally uses an **array** and **resizes automatically** when needed.

-

Differences: Array vs ArrayList

• Feature	• Array	• ArrayList
• Size	• Fixed	• Dynamic
• Type Support	• Can store primitives (int[])	• Stores Objects only (Integer, String)
• Syntax	• <code>int[] arr = new int[5];</code>	• <code>ArrayList<Integer> list = new ArrayList<>();</code>
• Methods	• No built-in methods	• Rich methods (add, remove, contains, etc.)
• Performance	• Faster (no method overhead)	• Slower due to method calls
• Flexibility	• Low	• High

ArrayList Example in Java:

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        ArrayList<Integer> list = new ArrayList<>();
        // Add elements
        list.add(10);
        list.add(20);
        list.add(30);
        // Access
        System.out.println(list.get(1)); // 20
        // Modify
        list.set(1, 25); // changes 20 to 25
        // Remove
        list.remove(0); // removes 10
        // Print all elements
        for (int val : list) {
            System.out.print(val + " ");
        }
    }
}
```



```

    }
}
}

```

When to Use Array vs ArrayList?

• Use Case	• Suggestion
• Fixed size data (e.g., 7 days of week)	• Array
• Need dynamic resizing	• ArrayList
• Want built-in methods (search, remove, etc.)	• ArrayList
• Performance-critical code (tight loops)	• Array
•	
Suggested Practice Problems	
• Problem	• Level
• Reverse an array	• Easy
• Check if array is sorted	• Easy
• Count even/odd numbers	• Easy
• Find second largest element	• Easy-Medium
• Difference between Array and ArrayList usage	• Conceptual
•	

DSA Lecture 1 – 15 Conceptual & Coding Questions

- 1 What is the difference between a Data Structure and an Algorithm?
1. Why do we need Data Structures in programming?
2. Name 5 commonly used data structures and one use case of each.

3. What is Time Complexity and why is it important?
4. Explain the difference between Best, Worst, and Average Case Time Complexities.
5. Tell the time complexities following algorithms:
 - ♦ Linear Search
 - ♦ Binary Search
 - ♦ Merge Sort
 - ♦ Bubble Sort
6. What is Space Complexity? How is it different from Time Complexity?
7. What is the time complexity of accessing an element in an array by index?
8. Explain the memory layout of an array. Why is array access $O(1)$?
9. What are the limitations of arrays in Java?
10. Write a Java method to reverse an array in-place.
11. What is the time and space complexity of the array reverse method?
12. How is an ArrayList different from an Array in Java?
13. What happens internally when an ArrayList exceeds its current capacity?
14. What is the time complexity of these ArrayList operations:
 - ♦ add(element)
 - ♦ get(index)
 - ♦ remove(index)

•

1. What is the difference between a Data Structure and an Algorithm?

• Aspect	• Data Structure	• Algorithm
• Definition	• A way of organizing and storing data	• A step-by-step procedure to solve a problem
• Purpose	• To manage and organize data efficiently	• To perform operations on data
• Example	• Array, LinkedList, Stack, Queue	• Binary Search, Merge Sort, Dijkstra's
•		

Analogy:

Think of **data structures as containers (boxes)** and **algorithms as the process (recipe)** to use those containers.

2. Why do we need Data Structures in programming?

Because they help in:

- **Efficient data storage and retrieval**
 1. **Reducing time and space complexity**
 2. **Choosing the right structure for specific problems**

•

Example:

1. Using a **HashMap** gives average $O(1)$ lookup for key-value pairs.

•

3. Name 5 commonly used data structures and one use case of each.

• Data Structure	• Use Case
• Array	• Storing fixed-size elements (e.g. months)
• LinkedList	• Insertion/deletion at dynamic positions
• Stack	• Undo operation in editors (LIFO)
• Queue	• Task scheduling (FIFO)
• HashMap	• Storing key-value pairs (e.g. dictionary)

4. What is Time Complexity and why is it important?

Time Complexity is a measure of the amount of **time an algorithm takes** to complete as a function of input size n .

Importance:

- Helps evaluate algorithm efficiency
 1. Predicts scalability
 2. Compares multiple approaches

•

5. Difference between Best, Worst, and Average Case Time Complexities

• Case	• Description	• Example (Linear Search)
• Best Case	• Fastest scenario	• First element matched $\rightarrow O(1)$
• Worst Case	• Slowest scenario	• Last element or not found $\rightarrow O(n)$

• Average Case	• Expected scenario over random inputs	• Element somewhere in middle → $O(n/2) \approx O(n)$
•		

6. Time Complexities of Algorithms

• Algorithm	• Time Complexity
• Linear Search	• $O(n)$
• Binary Search	• $O(\log n)$ (sorted array only)
• Merge Sort	• $O(n \log n)$
• Bubble Sort	• $O(n^2)$
•	

7. What is Space Complexity? How is it different from Time Complexity?

- **Space Complexity:** Amount of **extra memory** used by an algorithm.
 1. **Time Complexity:** Measures **execution time** as input size grows.
- **Example:**
 1. A recursive Fibonacci has $O(n)$ space due to call stack.
 2. An iterative Fibonacci has $O(1)$ space.
-

8. What is the time complexity of accessing an element in an array by index?

$O(1)$ – Constant time

Because memory is **contiguous**, index i is computed as:

$\text{address} = \text{base_address} + i * \text{element_size}$

9. Explain the memory layout of an array. Why is array access $O(1)$?

1. In Java, arrays are stored **contiguously** in memory.
 2. Access is $O(1)$ because:
 - ♦ You directly calculate the memory offset from the base.
- **Example:** Accessing `arr[4]` involves 1 step (no iteration needed).

10. What are the limitations of arrays in Java?

1. **Fixed Size** – Cannot grow or shrink after creation.
2. **Insertion/Deletion** – Expensive ($O(n)$).
3. **Homogeneous** – All elements must be of the same type.
4. **Wastage** – May lead to memory waste if not fully used.

•

11. Write a Java method to reverse an array in-place.

```
public class ArrayUtils {
    public static void reverseArray(int[] arr) {
        int left = 0, right = arr.length - 1;
        while (left < right) {
            // Swap arr[left] and arr[right]
            int temp = arr[left];
            arr[left] = arr[right];
            arr[right] = temp;
            left++;
            right--;
        }
    }
    public static void main(String[] args) {
        int[] arr = {1, 2, 3, 4, 5};
        reverseArray(arr);
        for (int num : arr) {
            System.out.print(num + " "); // Output: 5 4 3 2 1
        }
    }
}
```

12. What is the time and space complexity of the array reverse method?

1. **Time Complexity:** $O(n)$
2. **Space Complexity:** $O(1)$ → In-place, no extra array used

•

13. How is an ArrayList different from an Array in Java?

• Feature	• Array	• ArrayList
• Size	• Fixed	• Dynamic (auto-resizable)
• Type	• Can be primitive	• Objects only (uses wrapper types)
• Flexibility	• Less flexible	• More flexible (methods, resizing)
• Syntax	• <code>int[] arr</code>	• <code>ArrayList<Integer> list</code>

ArrayList is built over arrays, manages capacity and resizing internally.

14. What happens internally when an ArrayList exceeds its current capacity?

- A **new larger array** is created (usually 1.5x or 2x current size).
 1. All old elements are **copied** to the new array.
 2. Reference is updated to new array.
- This is why add() is **amortized O(1)** but may occasionally take **O(n)**.

15. Time complexity of ArrayList operations

• Operation	• Time Complexity	• Explanation
• add(element)	• Amortized O(1)	• Adds to the end; occasional resize is O(n)
• get(index)	• O(1)	• Direct index access like arrays
• remove(index)	• O(n)	• Needs to shift elements after index

•